Reading Handwritten Arabic with Small Vision-Language Models Under 3 GB

Quantization, LoRA, sidecar SLMs, tokenizers and benchmarks for a 4-bit Qwen3-VL on one 24 GB GPU

Technical write-up · October 2026 · Cheque-processing IDP project

ABSTRACT

For several months I worked on an on-prem cheque-extraction system built around a 4-bit Qwen3-VL-8B served by vLLM on a single NVIDIA A10, with about 3 GB of VRAM left for anything new. Printed fields were straightforward. The handwritten Arabic payee line was not, and it became the lens for this post: how AWQ, GPTQ, FP8 and NF4 differ in practice, why merging a LoRA and re-quantizing gives you a *different model*, how a 95/5 data mix taught the model to forget its job, why a base model's pre-training leaks through a fine-tune, why self-consistency cannot fix systematic perception errors, and why a scorer has to be frozen before the experiment. It closes with where the work is strong and where it is still thin.

Table of contents

0. The box I had to work in
1. Anatomy of the model
2. Tokens: the text kind and the pixel kind
3. Quantization: where the deployed model actually lives
4. Fine-tuning: LoRA, QLoRA, and the model that forgot its job
5. Pre-training: the prior you inherit decides everything
6. Small language models and the 3 GB sidecar
7. Distillation under a data-residency constraint
8. RLHF, DPO, GRPO for a perception task
9. Deduplication and leakage on a tiny dataset
10. Benchmarking: the part that taught me the most
11. Serving: vLLM, SGLang, and multi-LoRA
12. Triton and the kernels underneath
13. Distributed training
14. Experiment tracking and MLflow
15. The dead-path ledger
16. Strengths, limits, and principles I'd carry forward
17. References

0. The box I had to work in

The model, the GPU and the deployment target were fixed before the first experiment:

- **Hardware:** one NVIDIA A10 (24 GB, Ampere, no FP8 tensor cores). The production VLM and its KV cache used about 18 GB of it, leaving **3–3.5 GB** for anything extra.
- **Data residency: no external APIs.** Cheques are regulated financial documents.
- **Base SLA:** `Qwen3-VL-8B-Instruct`, AWQ 4-bit, on vLLM. It already handled printed English and structured fields well, and those could not get worse.

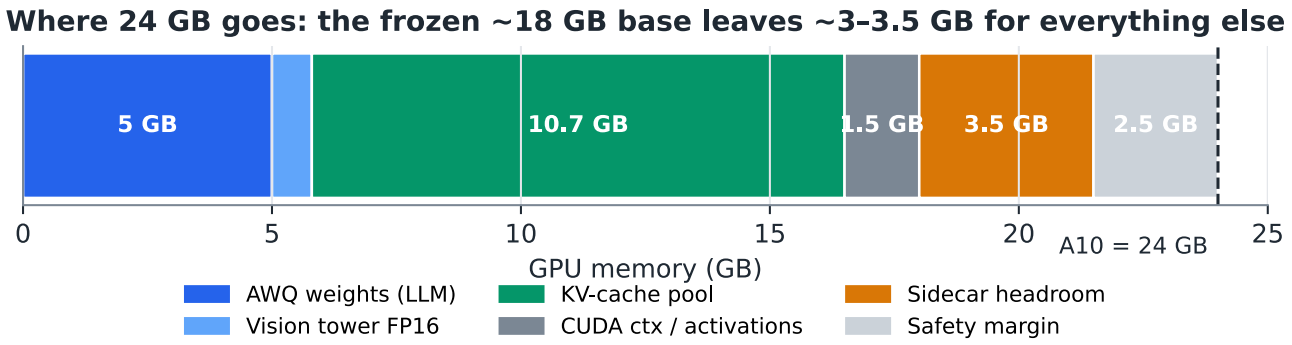


Figure 1. VRAM on the A10. The frozen base uses ~18 GB; anything new has to fit in ~3–3.5 GB.

I kept a VRAM spreadsheet for the whole project. Every candidate got a row: weights at the served precision, the FP16 vision tower, KV cache at the context I actually needed, CUDA context and graphs, and whatever was left against the 3 GB line. It was the fastest filter I had: a row that didn't close ended an idea before any download.

With the base frozen and VRAM fixed, the design space is not "which model" but what you can attach to the model you have: change the weights (LoRA → merge → re-quantize), change what the model sees (crops, layout, preprocessing), or add a small specialist beside it (the "sidecar").

> **DECISION LOG**
>
> **Deterministic before generative.** IBAN, MICR and every checksum were taken away from the VLM from day one. A VLM produces *plausible* digits; an IBAN has an ISO-7064 mod-97 checksum and a MICR line a fixed E-13B font. When code can verify a field, code reads it and the VLM is the fallback. This rule did more for the pipeline than several of the model experiments below.

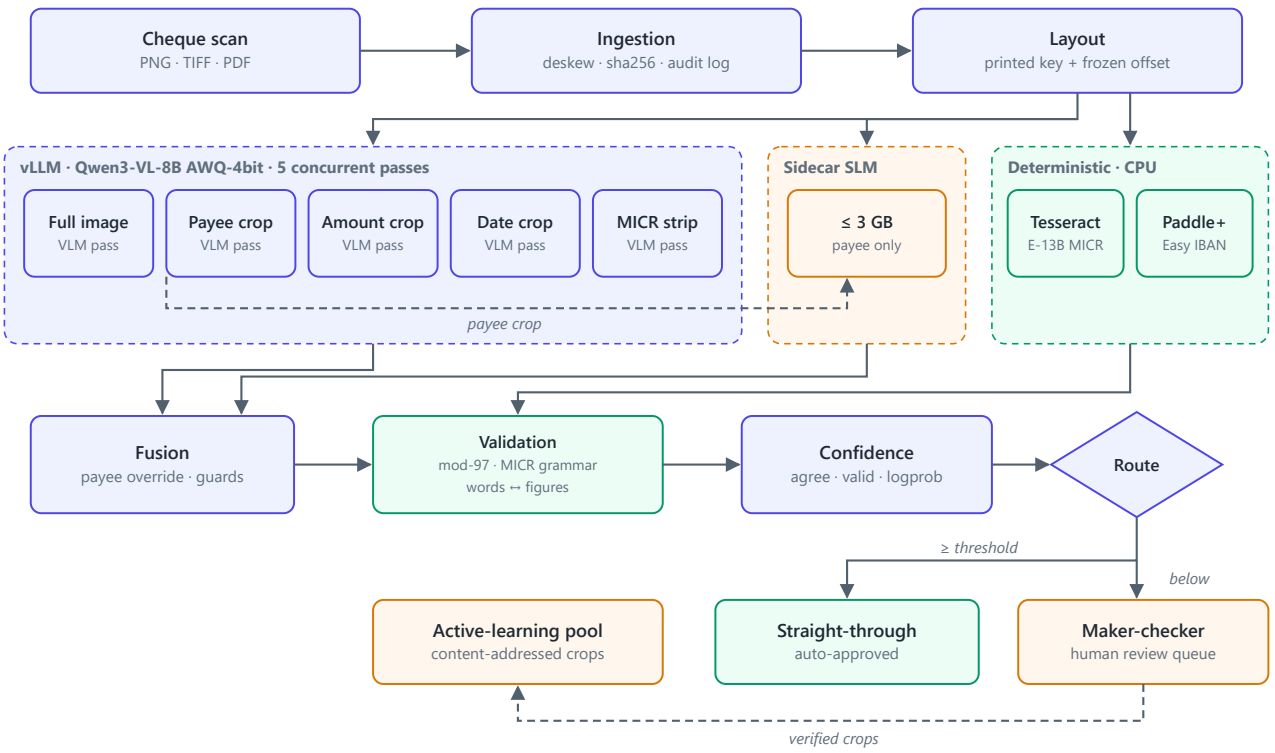


Figure 2. The pipeline. Five concurrent VLM passes and CPU-side deterministic engines feed fusion, validation, confidence and routing. Green boxes are verifiable by code; amber boxes are the parts that learn.

1. Anatomy of the model

Qwen3-VL follows ViT → merger → LLM, with no cross-attention. Three details mattered for OCR:

- **The merger is a learned contract.** I wanted to transplant the vision encoder from an Arabic-handwriting fine-tune of a smaller Qwen. It can't work: the merger was trained jointly with its own ViT, and across sizes the shapes don't even line up (`expected [128, 2048], got [128, 4096]`). Where the vision side must change, the move is **LoRA on the ViT and merger together**, not a transplant.
- **DeepStack** injects intermediate ViT features (layers 8/16/24) into the first decoder layers without lengthening the sequence. That is where OCR detail lives, and why heavy preprocessing can hurt: it shifts low-level statistics that DeepStack passes straight into the LLM.
- **M-RoPE** encodes time, height and width, so a token's position is *where it was on the page*. A tight crop changes spatial context, not just resolution.

Qwen3.5 is natively multimodal (early fusion) with a hybrid **Gated DeltaNet + gated attention** stack. DeltaNet layers keep a fixed-size recurrent state instead of a growing KV cache, so memory scales differently with sequence length. It was the first time a change in *architecture*, rather than a change in parameter count, showed up in my VRAM spreadsheet.

> FIELD NOTE
>
> **The MoE VRAM trap.** A 35B-A3B mixture-of-experts model "only uses 3B parameters per token", so on paper it fits. The spreadsheet says otherwise: every expert must be resident, 35B × 4 bits ≈ 17.5 GB before the vision tower and KV cache. **MoE saves FLOPs, not memory.**

2. Tokens: the text kind and the pixel kind

Pixel tokens are the real context budget. Qwen-VL resizes images to multiples of 32 px and the decoder sees `(H/16)·(W/16)/4` visual tokens: ~1,750 for a 2000 × 900 cheque, ~350 for a payee crop. The first QLoRA run inherited `max_seq_length=2048` from a text recipe and stopped 16% of the way through when large scans overflowed it. In a VLM, **`max_seq_length` is a function of image resolution**. The same arithmetic explains latency: a crop is 5× fewer prefill tokens, so five parallel crop passes cost about one full-image pass.

Fertility and byte fallback. Qwen's ~151k byte-level BPE has real Arabic merges. GLM-OCR, a sidecar candidate, has a Chinese-English decoder; a four-word Arabic name falls back to 20+ byte tokens, each a fresh chance for the language prior to override the pixels. If the target script costs 3–4× the tokens of English, you are fine-tuning against the vocabulary, and vocabulary adaptation (Yamaguchi et al., 2024) is the real fix.

Tokenizer contracts. A Qwen3.5-0.8B fine-tune shipped a tokenizer class vLLM didn't recognise; serving with the base tokenizer is safe only if the vocabulary is unchanged. HATFormer flips the image horizontally before encoding because its decoder emits RTL text in logical order; unflipped input still yields fluent, wrong Arabic. **Preprocessing is part of the tokenizer contract.**

Normalization (alef variants → ا, ة/ه, ى/ي, tashkeel, tatweel) sits between tokens and metrics. Too little punishes orthographic choices; too much grades generously. §10 returns to this.

3. Quantization: where the deployed model actually lives

This is where I went deepest, because it is where the model you evaluate quietly stops being the model you serve.

Format	Idea	What it meant in practice
GPTQ	Quantize column by column; the inverse Hessian from calibration activations pushes each column's rounding error onto columns not yet quantized	Trainable with LoRA (GPTQModel + PEFT). My GPTQ fine-tunes served **~3× slower** than the AWQ base on the same vLLM
AWQ	~1% of input channels are salient because their *activations* are large; scale them up before quantizing, `s = mean(|x|)^α`	**No backward pass** (AutoAWQ is archived), so no LoRA training on it
FP8 (E4M3)	8-bit float with per-tensor/channel scales	Ampere has no FP8 tensor cores: memory savings, no compute speed-up
NF4	16 levels at the quantiles of N(0,1), double-quantized scales	What QLoRA *trains* against. Not what you serve

The vision tower stays in 16-bit, and the reason is arithmetic. llm-compressor's W4A16 recipe died on `model.visual.blocks.*.mlp.linear_fc2`: the ViT MLP width is **4304**, and **4304 mod 128 = 80**, so a 128-wide group can't tile it. It's my favourite bug of the project: one line of modular arithmetic explains a crash, a community convention and a quality argument at once. MBQ (CVPR 2025) shows language tokens are far more quantization-sensitive than vision tokens, and the ViT is BF16 in *both* the NF4 training checkpoint and the AWQ checkpoint, so a vision-side LoRA has zero cross-format mismatch.

```
ignore = ["lm_head", "re:model\\.visual\\..*", "re:model\\.merger\\..*"]   # llm-compressor >=
        0.10: regex
recipe = QuantizationModifier(targets="Linear", scheme="W4A16", ignore=ignore)
```

Calibration data is a hyperparameter. `oneshot()` runs happily with no calibration set, and many recipes reach for `ultrachat_200k`, which is English chat. For a model whose weak spot is Arabic handwriting, the channels that matter never light up during calibration, so AWQ doesn't protect them. I replaced the default with AWQ calibrated on 256 chat-templated cheque samples, and **AutoRound** (learned rounding and clipping by signed gradient descent) on a 50/50 cheque/ `pile-10k` mix. Calibrate in the chat template you serve with.

3.1 The triple mismatch

The most important thing I learned about QLoRA, measured on two separate fine-tunes:

```
train:    LoRA fits ΔW against dequant_NF4(W)    # mismatch 1: dequant_NF4(W) ≠ W
merge:    W' = W_fp16 + (α/r)·BA                # ΔW fitted to NF4 noise, applied to clean W
serve:    quant_GPTQ(W')                         # mismatch 2: new quantization of new weights
compare:  vs. base AWQ(W)                        # mismatch 3: other algorithm, other grid
```

The deployed model is **not** "base plus my adapter"; it is a different model, quantized by a different algorithm from different weights, and every field has to be re-certified. A narrow-domain LoRA adds outliers that AWQ's salient-channel protection never saw (RoLoRA documents this collapse); a rank-128 update pushes many weights across 4-bit bin boundaries; and part of the adapter learns to undo NF4

artefacts absent from the FP16 weights it is merged into. The fixes, in order: **don't merge** (serve the adapter, §11); **LoftQ** (initialize LoRA to absorb quantization error); **QA-LoRA** (merge into the quantized grid); **quantization-aware distillation** (§7).

3.2 Was precision even the bottleneck?

QARI-OCR reports Arabic CER rising by over an order of magnitude from 8-bit to 4-bit, the obvious root-cause candidate, so I tested it. The FP8 build of the same base was only marginally better on Arabic handwriting, and the two builds disagreed on individual cheques in *both* directions. Quantization damage and the 8B base's limited Arabic-handwriting perception were **two compounding bottlenecks**; a 3B specialist showed no BF16-vs-4-bit difference at all. When the task is mismatched, precision is second-order.

> FIELD NOTE
> **"weights on disk" ≠ "VRAM at runtime".** A 4B model at 4 bits is "about 2.5 GB" on the Hub card. Served by vLLM it reported **5.58 GiB**: FP16 vision tower, activation workspace, CUDA graphs, KV blocks. The only number that counts is the one in the server log after warm-up.

4. Fine-tuning: LoRA, QLoRA, and the model that forgot its job

4.1 The mechanics that matter

LoRA learns `h = Wx + (α/r)·BAx` with `B = 0` at init. **α/r is an effective learning-rate multiplier**, which is why recipes move α with r. **rsLoRA** uses α/√r, so turning it on with an LR tuned for α/r is a larger step; in one recipe I studied it diverged the run. **QLoRA** stores W in NF4 and computes in BF16 with paged 8-bit AdamW. I trained with Unsloth's `FastVisionModel` (cosine schedule, 10% warmup, `adamw_8bit`, effective batch 8) and kept a plain HF + PEFT path for when version pins fought with TRL.

Freeze the vision tower or not? Forgetting research favours freezing; every successful Arabic-handwriting recipe (AtlasOCR, QARI-OCR, Ketaba-OCR) unfreezes it, and AtlasOCR's ablation shows frozen vision clearly worse. The resolution is task-specific: **unfreeze when the target is a new script or handwriting style, freeze when it is a new output format.** rsLoRA showed the same split (off for printed text, on for handwriting). Hyperparameters transfer within a task regime, not across it.

4.2 V14 and V15: two fine-tunes, two opposite failures

V14 mixed real cheque samples with a few thousand Arabic OCR samples: **~5.6% cheque, 94.4% generic Arabic OCR**, warm-started from a public Arabic-manuscript LoRA (r=32, α=64, LR 1e-4, A100). It produced the best handwritten-payee reading so far, and the structured fields (bank, drawer, IBAN) **collapsed**. Generic OCR teaches "output all the text you see"; cheque extraction is "output these eight fields in this schema". At 94% of the gradient, the model kept the knowledge and lost the *instruction*.

V15 was the opposite pole: 100% cheque data, three prompt variants, three epochs, re-quantized to GPTQ. Structure came back; Arabic payee reading fell *below the untouched base*; latency tripled; and the model began writing **memorized training-set names** onto unrelated cheques.

Then came the finding I'm proudest of from the fine-tuning work. When I diffed the two runs cheque by cheque, **the cheques V14 improved and the cheques V15 improved were completely disjoint.** V14 had learned to *read Arabic*; V15 had learned to *fill the form*.

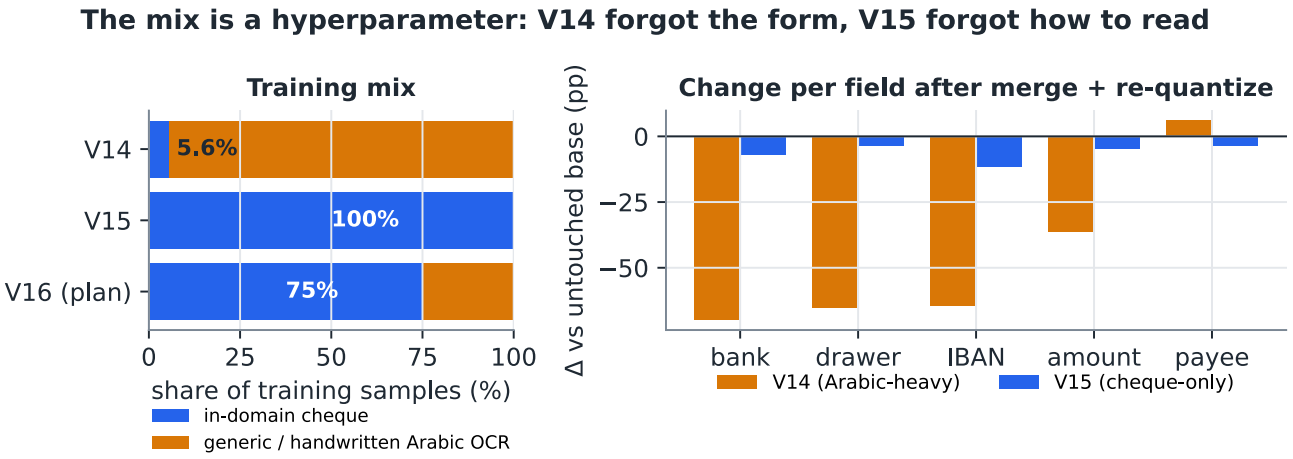


Figure 3. Left: the training mixes. Right: per-field change versus the untouched base after merge and re-quantization (83-cheque benchmark).

DECISION LOG

Fix the ratio, not the ingredients. The wrong conclusion is "Arabic data hurts". The mixing ratio is a first-class hyperparameter, and a monolithic merged adapter lets one bad ratio damage every field. V16 targets ~70–80% cheque data plus ~20–30% *handwritten* Arabic only. The deeper fix is architectural: a payee adapter active only on the payee-crop pass (§11).

4.3 Failure modes in the fine-tuning stack

- **Labels teach conventions.** A fine-tune began emitting English transliterations that aren't on the cheque. The trace led to ground truth annotated as `"<Arabic> / <English>"`; a five-line split that keeps the script actually written fixed more than any hyperparameter. A label audit is part of the training recipe.
- **The untied `lm_head`.** A popular Arabic OCR checkpoint ships no `lm_head.weight` but sets `tie_word_embeddings: false`, so Transformers randomly initializes the head and the model emits random Unicode, indistinguishable from "can't read this". Re-tying turned noise into Arabic and cut latency ~8×. **A base-model sanity eval is a hard gate before any training run.**
- **Loss on the prompt.** A collator that masks only padding spends most of the gradient on the prompt and image placeholders. Mask everything before the assistant turn (`labels[:, :n_prompt] = -100`).

5. Pre-training: the prior you inherit decides everything

The most surprising result of the project was a *pre-training* result.

Two community fine-tunes of **GLM-OCR** (CogViT + GLM-0.5B), trained on more Arabic data than the Qwen-based specialist I compared them with, were dramatically worse. On Arabic payee crops they produced fluent Chinese, HTML tables around a few Arabic characters, Chinese HTML comments (`<!-- 省略中间20行 -->`, "omitting the middle 20 lines"), empty `<think></think>` tags and runaway repetition. The

first time a handwritten Arabic name came back as a Chinese HTML comment, I stopped reading outputs as scores and started reading them as evidence about a corpus I would never see. The model had been pre-trained on Chinese forms and scraped HTML; given a bank form it couldn't read, **it fell back to the most likely bank form in its pre-training distribution.**

> **DECISION LOG**
> **Base selection beats data volume.** A fine-tune moves you within the basin pre-training put you in; it doesn't move you to a new one. `bad_words_ids` over CJK, regex-stripping HTML and `repetition_penalty` are band-aids, and the fine-tune I tested already stripped HTML and still failed.

What continued pre-training would take. HATFormer's recipe is the clearest: start from a printed-English TrOCR, continue pre-training on ~1M synthetic Arabic lines, then fine-tune on real handwriting; ablating the synthetic stage is what breaks it. I fully fine-tuned HATFormer (334M, differential LR 1e-5 encoder / 5e-5 decoder) on my crops and it still failed, because its real-data stage was historical manuscripts. *Domain* and *script* are separate axes. The synthetic generator becomes the critical path, and Arabic punishes it: without **HarfBuzz shaping** you render isolated glyphs, effectively a different script; and a printed-overlay set I built (45 names × 82 cheque backgrounds) made the sidecar *worse* on real handwriting. The lever is **handwritten pixels** (GAN/diffusion handwriting synthesis such as ScrabbleGAN forks), not printed ones.

6. Small language models and the 3 GB sidecar

With the base frozen, the natural architecture is a **small specialist** that reads one field and a fusion rule that decides when to trust it. Inside its domain, small and fine-tuned beats big and general.

Candidate class	Fit
7B Arabic VLMs	~4.2 GB even at 2-bit GGUF with the vision projector
2–4B VLM specialists	The sweet spot. The 4B read best with preprocessing but served at **5.58 GiB**; the 2B fits
0.8B VLM fine-tune	Fits, but ~400k *wrong-domain* samples (printed, religious, Persian) made it regress; a small model has no spare capacity to hold both
TrOCR-class, clean-line engines	~0.7 GB, but need a clean segmented line; payee crops carry template text, rules and stamps

The task is not "read Arabic text"; it is "isolate the handwritten name from a crop full of printed template text". Small VLMs **transcribe everything they see**, which is precisely the wrong behaviour.

Preprocessing is model-dependent. I built a classical-CV crop cleanup (HSV ink mask, connected components classified by stroke-width variance from a distance transform, template-zone removal, descender-safe guideline removal). CLAHE + unsharp masking then **helped one sidecar, did nothing for another and destroyed a third**, by making printed template text more salient. Tune preprocessing per model.

Fusion and the inert sidecar. A three-way vote among voters with *correlated* errors is worse than the best voter, so fusion became an **asymmetric override**:

```
if base_is_arabic and sherif_ok:
    disagree  = not fuzzy_name_match(base, sherif_payee)
    conf_ok   = (sherif_conf is not None) and (sherif_conf >= 0.92)
    not_payer = not (payer_name and word_overlap(sherif_payee, payer_name) > 0.8)
    if disagree and conf_ok and not_payer:
        return sherif_payee, "v7m_sherif_override"
return base, "v7m_vlm"
```

A decision-path audit I ran showed **zero overrides**: confidence was hard-coded to `None` at the call site. Zero is a number you only notice once you start counting, and that counter is now the cheapest monitoring in the pipeline.

---

7. Distillation under a data-residency constraint

The frontier reference read handwritten Arabic better than anything on-prem, but cheques can't leave the premises, so it can't be the teacher. The design I'd use: an on-prem sibling with the **same tokenizer** (Qwen3.5-9B, or a 27B on a multi-GPU node) in BF16 as teacher, the 2B sidecar as student, and token-level KD on answer tokens:

```
L = α · CE(student, y_gt) + (1−α) · T² · KL(softmax(z_t/T) ‖ softmax(z_s/T))
```

The KL term carries the teacher's uncertainty between letters that differ by one dot (ج/ڊ/ح), exactly what a hard label discards; T^2 keeps gradient magnitudes stable as T changes. Sequence-level KD (Kim & Rush) is the cheaper fallback and can label unlabeled crops. **Quantization-aware distillation** into the already-quantized student is the most principled answer I found to the triple mismatch.

---

8. RLHF, DPO, GRPO for a perception task

OCR has one right answer, but SFT left **specific wrong answers with high likelihood**: the printed template phrase, the drawer's name in the payee slot, memorized training names. Cross-entropy only lifts the right answer indirectly; **DPO pushes down the specific wrong ones**:

```
L_DPO = −log σ( β · [ (log π(y_w|x) − log π_ref(y_w|x)) − (log π(y_l|x) − log π_ref(y_l|x)) ] )
```

Chosen = ground truth; rejected = self-mined hard negatives (sample the SFT model at $T > 0$, keep reads the frozen scorer marks wrong, add the drawer name); reference = the adapter-disabled base, so no second model in VRAM; $\beta \approx 0.1$, IPO if negatives are noisy.

GRPO with a verifiable reward (`1 − CER`) needs no reward model. But RL optimizes whatever the reward doesn't penalize; GRPO is the likely source of GLM-OCR's `<think>` tags. On this task chain-of-thought **hurt** reading (temperature 0, no reasoning was best), so any reward should penalize every token that isn't the answer. *OCR is perception, not reasoning*.

9. Deduplication and leakage on a tiny dataset

Cheques are *structurally* near-duplicates: same template, same printed phrases, different handwriting. A random split puts the same template on both sides.

- **Group K-fold by cheque ID**, so every prompt variant and jittered crop of a cheque stays on one side.
- **Path-level template exclusion** for the overlay set. Basenames like `image6.png` repeat across folders, so the filter matches the normalized full path and **asserts** no validation background was selected.
- **A content-addressed, frozen eval set.** Every eval crop is SHA-256 hashed into a manifest with a `set_hash`; trainers drop those IDs and benchmarks warn if the hash changes.
- **Leave-one-out cross-bank scoring** for layout.
- **Near-duplicate detection:** 64-bit **pHash** (Hamming ≤ 8 = same layout), **MinHash-LSH** over 3-grams of Arabic-normalized labels, and union-find over both graphs feeding `GroupKFold`.

Deduplicate on the axis your model can memorize; for a VLM on forms, that is the template, not the file.

10. Benchmarking: the part that taught me the most

On a small, multilingual eval set, measurement is where most of the leverage lives, and most of the risk.

The instrument. Field-level scorers (amounts as floats, dates as ISO with day/month swap detection, bank aliases, MICR digit containment, normalized IBAN, fuzzy names), **ANLS** ($\tau = 0.5$) at corpus level, oracle sidecar scoring on manual crops, and a business-weighted composite. ANLS scored correct bank names 0.0 because ground truth held the brand and the model the legal name: **a string metric and a domain metric can disagree completely**.

10.1 Measurement traps, and the control for each

1. **Small-N granularity.** An early 4-bit vs 8-bit comparison ran on 18 cheques, 8 carrying the field. One flip moved the score **12.5 points**, and the gap vanished at 83. That number is why I compute 1/N before I look at any delta.
2. **Scorer leniency.** The fuzzy matcher accepts ≥ 50% word overlap; an Arabic company suffix (ذ م م) is three one-character tokens, so a wrong company could pass on the suffix alone. *Audit what the scorer accepts.*
3. **Scorer overfitting.** Adding normalization *to the scorer* lifted a field overnight with no model change; I reverted it. **Never change the scorer to improve accuracy.**
4. **Scorer drift.** Training and inference used different name matchers. *Vendor one scorer verbatim into every eval.*
5. **Proxies.** FP8 does not predict AWQ (§3.2). *Benchmark the exact checkpoint and flags you deploy.*
6. **Projections.** Literature-based projections looked tidy; tested on this task, none survived and several had the wrong sign (Figure 5). *Every number is* `VALIDATED` *with a report path, or* `UNVALIDATED`.
7. **Confounds.** The best Arabic result changed the model and the crops at once; it stays confounded until both ablations are in.

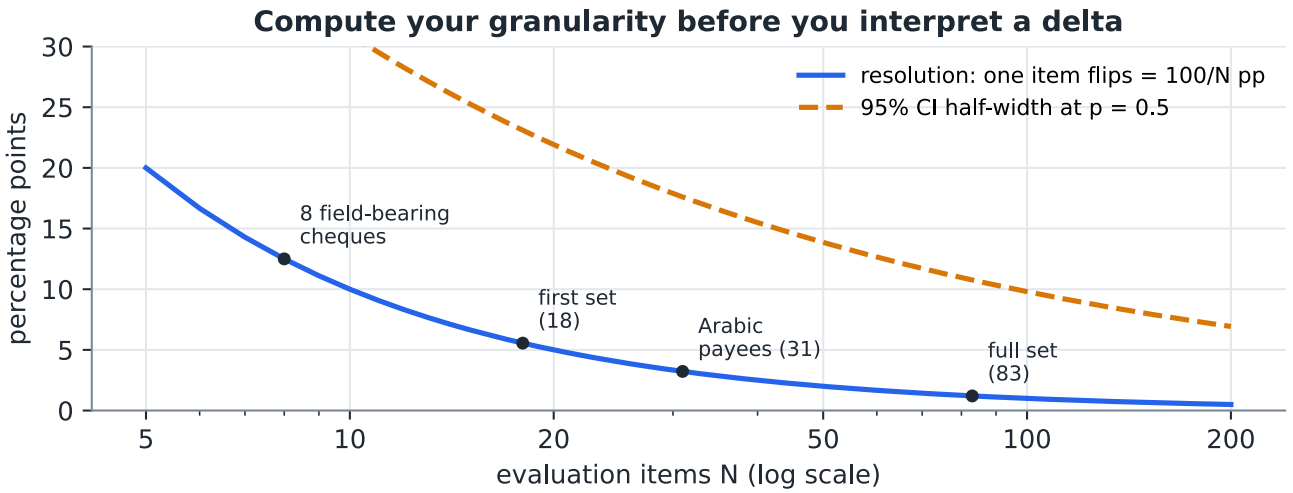


Figure 4. Resolution and worst-case 95% confidence half-width versus evaluation size.

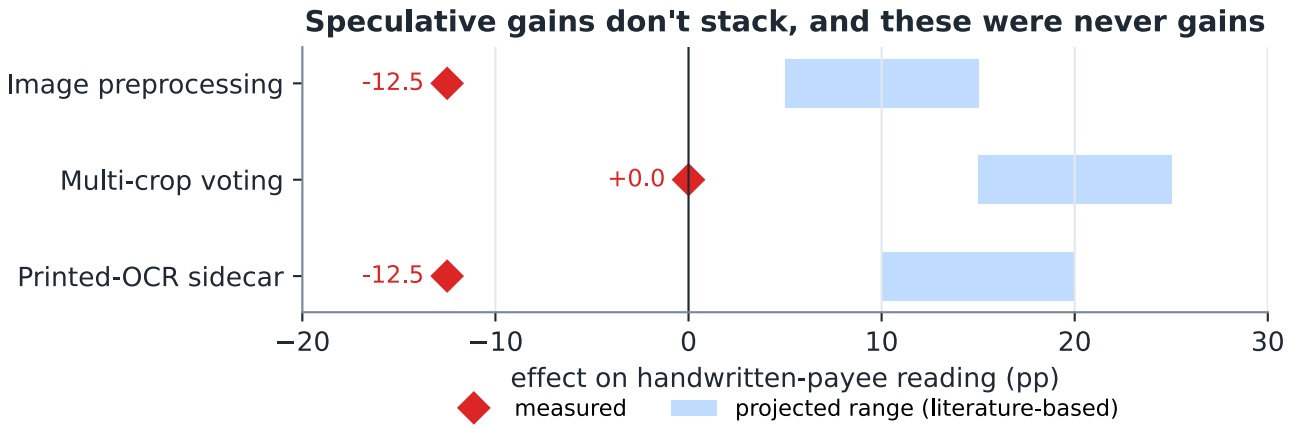


Figure 5. Literature-based projections versus what was measured on the same task.

10.2 Failure taxonomy before fixes

Bucketing the Arabic payee misses: **~two-thirds completely wrong reads** (fluent names not on the cheque), **~one-fifth character-level corruption** (dotted letters swapped), **~one-seventh template text**. That table ruled out three directions before they cost GPU hours:

- **Post-OCR correction** (spelling, translation, NER, perplexity) assumes a noisy channel. Most errors were the wrong string, and fluent ones that perplexity rates highly.
- **Self-consistency.** A published multi-view consensus method for printed text abstained on two-thirds of crops, and when views agreed they often agreed on the same wrong name. **A medoid of wrong answers is still wrong.**
- **Prompt optimization** stripped bilingual mixing, but the Arabic underneath was also wrong. It optimizes prompts, not perception.

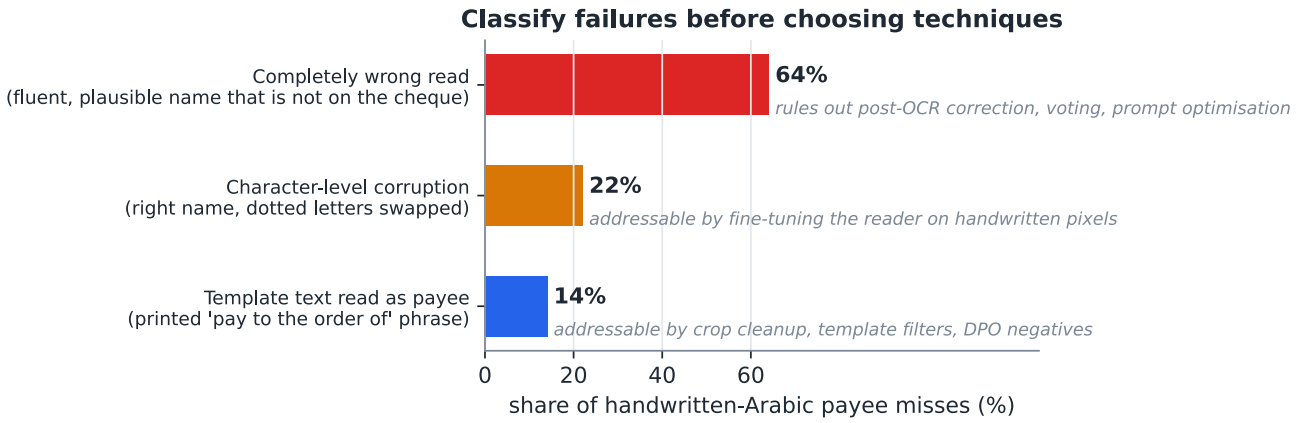


Figure 6. Taxonomy of handwritten-Arabic payee misses and which classes of fix each bucket rules out.

Adversarial verification. A refute-by-default review (~40 sources, 177 falsifiable claims, three skeptic votes each) killed several self-reported model-card numbers and produced the most useful sentence of the project: *no published system reaches 80% whole-name accuracy on free-form handwritten Arabic.*

Calibration is the real gate. For clearing, the objective is how many cheques can be auto-approved at a fixed error rate. Confidence blends pass agreement (0.4), deterministic validity (0.4) and `exp(mean token logprob)` (0.2); reliability curves and isotonic calibration turn it into a probability. Prefer false exceptions to silent passes.

11. Serving: vLLM, SGLang, and multi-LoRA

```
vllm serve cyankiwi/Qwen3-VL-8B-Instruct-AWQ-4bit --dtype float16 \
  --gpu-memory-utilization 0.92 --max-model-len 32768 --enable-chunked-prefill --block-size 16
```

AWQ kernels need FP16 activations. **PagedAttention** makes the KV cache a pool, and capping `--gpu-memory-utilization` is how the sidecar headroom in Figure 1 was carved out. **Chunked prefill** stops one full-resolution cheque from stalling every decode behind it. Five concurrent passes per cheque are batched continuously, so latency tracks the slowest pass. Generic JSON mode was safe; **schema-constrained decoding on the 4-bit model collapsed most fields to empty strings**: forced into a grammar, a weak model finds the shortest valid path.

Multi-LoRA in theory. S-LoRA/Punica keep one base in VRAM and apply a *different* adapter per request in the same batch (SGMV kernels). A payee adapter active only on the payee-crop request *can't regress fields it never touches*: the clean answer to V14/V15.

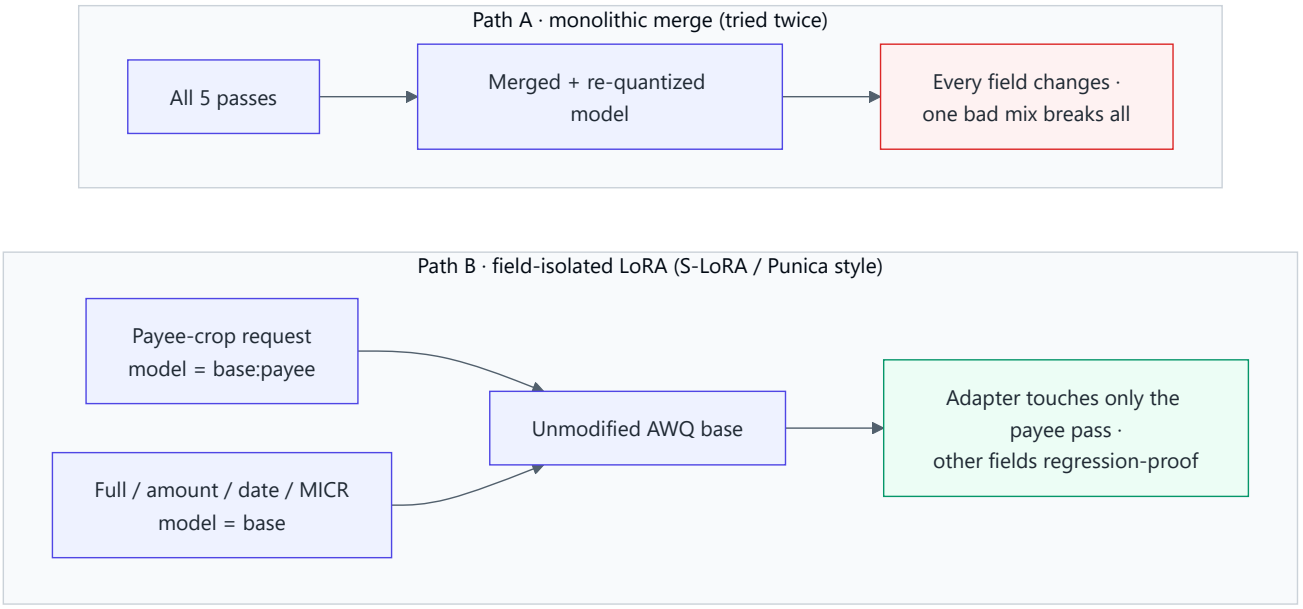


Figure 7. Monolithic merge versus field-isolated adapter serving.

In practice. vLLM's Qwen3-VL language LoRA crashed on `assert token_lora_mapping.size(0) == M` in the Punica shrink kernel: with multimodal inputs the token-to-adapter mapping is built over a different token count. SGLang added Qwen3-VL LoRA for the *language model only*, with per-request routing and dynamic loading. My first test collapsed every field to one memorized output: the adapter had been trained on vision *and* language layers and the server silently applied only half.

> **DECISION LOG**
>
> **Train for what you can serve.** If the server applies only `model.layers.*`, train with the vision tower frozen. "Unfreeze the ViT for handwriting" (§4.1) and "the server loads only language LoRA" are in direct tension; resolve it before the training run.

12. Triton and the kernels underneath

Almost every serving issue above was a kernel constraint: no AWQ backward pass, groups that must tile K, a Punica shape assertion, GPTQ vs AWQ kernel speed. Writing a GPTQ-layout W4A16 dequant kernel in Triton made them concrete:

```
@triton.jit
def _dequant_kernel(q_ptr, s_ptr, z_ptr, out_ptr, K, N, G: tl.constexpr,
                    BLOCK_K: tl.constexpr, BLOCK_N: tl.constexpr):
    offs_k = tl.program_id(0) * BLOCK_K + tl.arange(0, BLOCK_K)
    offs_n = tl.program_id(1) * BLOCK_N + tl.arange(0, BLOCK_N)
    mask = (offs_k[:, None] < K) & (offs_n[None, :] < N)
    packed = tl.load(q_ptr + (offs_k[:, None] // 8) * N + offs_n[None, :], mask=mask, other=0)
    q = ((packed >> ((offs_k[:, None] % 8) * 4)) & 0xF).to(tl.float16)
    g = offs_k[:, None] // G
    s = tl.load(s_ptr + g * N + offs_n[None, :], mask=mask, other=0.0)
    z = tl.load(z_ptr + g * N + offs_n[None, :], mask=mask, other=0.0)
    tl.store(out_ptr + offs_k[:, None] * N + offs_n[None, :], (q - z) * s, mask=mask)
```

`g = k // G` indexes a scale row, so a ragged last group breaks every tiling assumption; that is the 4304 mod 128 crash from §3, seen from the kernel's side. Decode-time W4A16 is memory-bound GEMV, so the win is **4× less weight traffic from HBM**, not faster math, which is why Marlin and the AWQ GEMM fuse dequantization into register tiles instead of materializing W. On Ampere, FP8 weights are upcast in-kernel: memory saving, no FLOPs.

---

13. Distributed training

DDP is enough for an NF4 8B base (~6 GB per rank). **FSDP + QLoRA** is what a distillation teacher needs: `bnb_4bit_quant_storage=bfloat16` lets FSDP shard 4-bit parameters, with decoder and vision blocks wrapped as separate units and `use_orig_params=True` for mixed frozen/trainable modules. **ZeRO-3** with CPU offload covers a full 3B fine-tune on cheap cards. The VLM-specific problem is **sequence-length variance**: the rank with the largest image sets the pace, so bucket by pixel area, and run `generate()`-based eval outside FSDP.

---

14. Experiment tracking and MLflow

Every number carries a `VALIDATED` / `UNVALIDATED` tag, and every result needs four identities pinned to it: **scorer identity** (two matchers give incomparable numbers), **code provenance**, **eval-set identity** (a 29- and a 30-crop version of the same set give two "oracle" numbers) and **checkpoint identity** ("AWQ" must resolve to one checkpoint plus server flags). In MLflow:

```
mlflow.log_params({"model": model, "quant": quant, "git_sha": _git_sha(), ...})
mlflow.log_param("eval_set_hash", manifest["set_hash"])
mlflow.log_param("scorer_hash", sha256(inspect.getsource(scorer_fn))[:16])
mlflow.log_dict(report, "report.json")
```

Promotion is a `champion` tag on the benchmark run; weights stay on the Hub, MLflow holds the evidence.

15. The dead-path ledger

Path	Why it died
Schema-constrained decoding on the 4-bit model	Grammar plus a weak model gives the shortest valid output
Merge LoRA → re-quantize (V14, V15)	The triple mismatch: a different model every time
Arabic-heavy mix (V14) / cheque-only mix (V15)	Forgot the task / memorized names
Swapping in an Arabic ViT	The merger is a joint contract with its ViT
Chain-of-thought, multi-view self-consistency	Perception errors are systematic, not noise
Global CLAHE/upscale	Model-dependent; amplifies template text
GLM-OCR fine-tunes	Base pre-training prior leaks through
0.8B specialist on 400k wrong-domain samples	Wrong-domain data degrades small models
HATFormer, clean-line OCR engines	Need clean segmented lines; manuscript domain
Printed-overlay synthetic fine-tune	Printed pixels don't teach handwriting
Post-OCR correction, translation, NER	Wrong strings, not noisy ones
YOLO on field values	Learns bank branding; replaced by anchor-relative layout
MoE 35B-A3B	MoE saves FLOPs, not VRAM

The layout entry hides the cleanest idea in the project: **the offset from a printed key ("Pay", "AED") to its handwritten value is a property of the cheque *convention*, shared across banks, while absolute position is a property of the bank's *branding*.** Anchor-relative boxes generalized across banks far better than absolute priors, yet barely moved Arabic reading: localization was decoupled from perception, which moved the effort back to the reader.

16. Strengths, limits, and principles I'd carry forward

16.1 Where this work is strong

- **Constraint-first systems design.** Every architecture decision traces back to ~3 GB of spare VRAM, no external APIs, and one rule: if code can verify a field, code reads it.
- **Root causes at the layer where they live.** A ViT width not divisible by the group size, a Punica shape assertion, an untied `lm_head`, a half-applied adapter, a fully resident MoE. Most cross two layers of the stack, which is where they are hardest to see.
- **Measurement that resists wishful thinking.** A frozen, content-addressed eval set, granularity before deltas, a scorer never tuned to raise a score, and a refute-by-default claim review.
- **Negative results with a mechanism attached.** The ledger explains *why* things fail on this distribution, so the next experiment is chosen rather than guessed.

16.2 Where it is still thin

- **The perception ceiling is not solved on-prem.** The frontier reference reads handwritten Arabic better and still misses a meaningful share, consistent with the literature.
- **The eval set is small.** 83 cheques cannot separate differences of a few points (Figure 4).

- **The best Arabic result is confounded** until the model-only and crop-only ablations are run.
- **Distillation, DPO/GRPO and multi-GPU training are not yet measured on this data.**
- **Confidence is provisional** until it is calibrated.

16.3 Principles I'd carry forward

1. **Build the benchmark first and freeze it.** Hash the eval set and the scorer; compute 1/N before any delta.
2. **Classify failures before choosing techniques.** The taxonomy decides which fixes are admissible.
3. **Respect the triple mismatch.** Serve the adapter, or use LoftQ, QA-LoRA or QAD.
4. **Treat data mix as a hyperparameter, and isolate skills** in per-field adapters.
5. **Pick the base for its pre-training distribution**: languages, tokenizer fertility, vision pre-training.
6. **Get handwritten pixels**: synthetic handwriting, active learning from exceptions, calibrated human review.

The project wasn't one long climb in accuracy. It was a steady shrinking of the hypothesis space, until what remained was the hard part: a 4-bit 8B model, on one card, perceiving handwriting in a script its pre-training barely covered. Everything around that core, from the quantizer's grid to the scorer's tolerance, is now understood well enough to say why it behaves the way it does.

---

17. References

Quantization

- Frantar et al., *GPTQ*, ICLR 2023 (arXiv:2210.17323)
- Lin et al., *AWQ: Activation-aware Weight Quantization*, MLSys 2024 (arXiv:2306.00978)
- Dettmers et al., *QLoRA*, NeurIPS 2023 (arXiv:2305.14314)
- Li et al., *LoftQ*, ICLR 2024 (arXiv:2310.08659); Xu et al., *QA-LoRA*, ICLR 2024
- Huang et al., *RoLoRA*, EMNLP 2024 Findings (arXiv:2407.08044)
- Li et al., *MBQ: Modality-Balanced Quantization for Large VLMs*, CVPR 2025 (arXiv:2412.19509)
- Intel, *AutoRound*; vLLM *llm-compressor*; Benjamin Marie, *Don't Merge Your LoRA Adapter Into a 4-bit LLM*

Fine-tuning and adaptation

- Hu et al., *LoRA*, ICLR 2022 (arXiv:2106.09685); Kalajdzievski, *rsLoRA* (arXiv:2312.03732)
- Yamaguchi et al., *Cross-lingual Vocabulary Adaptation* (arXiv:2402.10712)
- AtlasOCR; Ketaba-OCR-LoRA (NakbaNLP 2026); QARI-OCR (arXiv:2506.02295)

Vision-language models and OCR

- Bai et al., *Qwen2.5-VL Technical Report* (arXiv:2502.13923); Qwen3-VL / Qwen3.5 model cards
- GLM-OCR technical report (arXiv:2603.10910)
- Chan et al., *HATFormer* (arXiv:2410.02179); Li et al., *TrOCR* (arXiv:2109.10282)
- Fogel et al., *ScrabbleGAN*, CVPR 2020

Serving and systems

- Kwon et al., *PagedAttention* (vLLM), SOSP 2023 (arXiv:2309.06180)
- Sheng et al., *S-LoRA* (arXiv:2311.03285); Chen et al., *Punica* (arXiv:2310.18547)
- Zheng et al., *SGLang* (arXiv:2312.07104); Tillet et al., *Triton*, MAPL 2019
- Answer.AI, *FSDP + QLoRA*; Rajbhandari et al., *ZeRO*, SC 2020 (arXiv:1910.02054)

Distillation and preference optimization

- Hinton et al., *Distilling the Knowledge in a Neural Network* (arXiv:1503.02531)
- Kim & Rush, *Sequence-Level Knowledge Distillation*, EMNLP 2016 (arXiv:1606.07947)
- Rafailov et al., *DPO*, NeurIPS 2023 (arXiv:2305.18290); Azar et al., *IPO* (arXiv:2310.12036)
- Shao et al., *DeepSeekMath* (GRPO) (arXiv:2402.03300)

Evaluation

- Wang et al., *Self-Consistency*, ICLR 2023 (arXiv:2203.11171); multi-view consensus for OCR (arXiv:2603.19790)
- Biten et al., *Scene Text VQA* (ANLS), ICCV 2019; Khattab et al., *DSPy* (arXiv:2310.03714)
- Broder, *On the Resemblance and Containment of Documents* (MinHash), 1997